



**INTELLIGENT VACCINE COLD STORAGE MONITORING SYSTEM
INDUSTRY CONTEXT
TEST CASE DESIGN**

**Submitted in the partial fulfilment for the Course of
CSA1016 – SOFTWARE ENGINEERING**

**to the award of the degree of
BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY**

**Submitted by
SAMEER AHMED G (192421154)**

**Under the Supervision of
DR ANITA DAVAMANI**

**SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences**

Chennai-602105

August 2026

1. INTRODUCTION AND PROBLEM OVERVIEW

The Intelligent Vaccine Cold Storage Monitoring System is a full-stack application designed to monitor vaccine storage conditions and identify unsafe environmental conditions. The system collects temperature and humidity information from storage units and determines whether the recorded temperature is within the acceptable vaccine storage range.

The implemented prototype consists of a Flask REST API, a web-based monitoring dashboard, and MongoDB for storing sensor information. Docker and Docker Compose are used for deployment, while Git and GitHub are used for version control.

Testing is required to ensure that the system correctly processes valid sensor readings, identifies abnormal temperatures, handles invalid inputs, provides reliable API responses, maintains database connectivity, and displays the monitoring dashboard correctly.

The main testing objective is to verify the **correctness, reliability, functionality, usability, and robustness** of the system.

1.1 Testing Objectives

The testing process aims to:

- Verify all major functional requirements.
- Validate safe and critical temperature detection.
- Test normal and boundary temperature values.
- Verify invalid and incomplete sensor inputs.
- Validate REST API responses.
- Verify MongoDB connectivity.
- Test dashboard availability.
- Verify Docker-based application execution.
- Identify possible failure conditions.
- Ensure adequate requirement and test coverage.

2. REQUIREMENTS TO BE TESTED

2.1 Functional Requirements

Requirement ID	Functional Requirement
FR01	The system shall provide a web-based monitoring dashboard.
FR02	The system shall display vaccine storage units.
FR03	The system shall process temperature readings.
FR04	The system shall process humidity readings.
FR05	The system shall classify temperature as SAFE or CRITICAL.
FR06	The system shall generate/display critical alerts.
FR07	The system shall accept sensor readings through an API.
FR08	The system shall store sensor readings in MongoDB.
FR09	The system shall provide health-status information.
FR10	The system shall provide database-status information.

2.2 Non-Functional Requirements

Requirement ID	Non-Functional Requirement
NFR01	The system should respond to API requests within an acceptable time.
NFR02	The dashboard should be easy to understand and use.
NFR03	The system should handle invalid input safely.
NFR04	Sensor data should be stored reliably.
NFR05	The application should remain available when Docker services are running.
NFR06	The application should be maintainable through modular code organization.
NFR07	The system should support containerized deployment.
NFR08	The application should avoid exposing sensitive configuration information.

3. TEST SCENARIOS

Test scenarios represent the major situations that must be verified before considering the application reliable.

Scenario ID	Test Scenario	Type
TS01	Verify application root endpoint	Functional
TS02	Verify API health status	Functional
TS03	Retrieve all vaccine storage units	Functional
TS04	Retrieve a specific storage unit	Functional
TS05	Submit a valid sensor reading	Functional
TS06	Submit temperature within safe range	Functional
TS07	Submit temperature below safe range	Boundary/Negative
TS08	Submit temperature above safe range	Boundary/Negative
TS09	Test exact lower temperature boundary	Boundary
TS10	Test exact upper temperature boundary	Boundary
TS11	Submit missing temperature	Negative
TS12	Submit missing humidity	Negative
TS13	Submit missing storage-unit ID	Negative
TS14	Submit non-numeric temperature	Negative
TS15	Submit non-numeric humidity	Negative
TS16	Verify critical alerts	Functional
TS17	Verify MongoDB connectivity	Integration
TS18	Verify sensor data persistence	Integration
TS19	Verify dashboard availability	UI
TS20	Verify Docker containers	Deployment
TS21	Verify invalid storage-unit ID	Negative
TS22	Verify empty request body	Exceptional
TS23	Verify API response format	Functional
TS24	Verify application after container restart	Reliability
TS25	Verify dashboard usability	Usability

3.1 Main Testing Categories

The test scenarios are divided into four major categories.

Normal testing verifies that the system behaves correctly with valid inputs such as a temperature of 5°C and humidity of 50%.

Boundary testing verifies values at and around the safe temperature limits of 2°C and 8°C.

Invalid-input testing verifies how the system responds when required fields are missing or contain invalid values.

Exceptional-condition testing verifies conditions such as database connectivity problems, unavailable services, invalid requests, or container failures.

This combination provides broader coverage than testing only normal inputs.

4. DETAILED TEST CASE DESIGN

The following test cases are designed for the Intelligent Vaccine Cold Storage Monitoring System. Each test case contains the test scenario, execution steps, test data, expected result, actual result, and final status.

4.1 Functional Test Cases

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC01	Verify application root	Open / in browser/API client	GET /	Application status is displayed	Application status displayed	PASS
TC02	Verify health API	Send GET request	/api/health	Health status returned successfully	Healthy response received	PASS
TC03	Retrieve storage units	Send GET request	/api/storage-units	Storage-unit list displayed	Storage units returned	PASS
TC04	Retrieve individual unit	Send GET request	VACCINE-01	Unit information returned	Unit information returned	PASS
TC05	Submit valid sensor reading	Send POST request	5.2°C, 54%	Reading accepted	Reading accepted	PASS
TC06	Verify safe temperature	Submit sensor reading	5°C	Status = SAFE	SAFE returned	PASS
TC07	Verify critical temperature	Submit sensor reading	10.5°C	Status = CRITICAL	CRITICAL returned	PASS
TC08	Retrieve alerts	Send GET request	/api/alerts	Alert information returned	Alert information returned	PASS
TC09	Check database status	Send GET request	/api/db-status	Database connected	MongoDB connected	PASS

5. BOUNDARY VALUE ANALYSIS

Boundary Value Analysis is applied to the vaccine temperature requirement because the system has a defined safe operating range.

The safe range is:

$$2^{\circ}\text{C} \leq \text{Temperature} \leq 8^{\circ}\text{C}$$

Values immediately below, at, and immediately above the boundaries are tested.

TC ID	Temperature	Boundary Condition	Expected Result	Status
TC11	1.9°C	Just below lower boundary	CRITICAL	PASS
TC12	2.0°C	Lower boundary	SAFE	PASS
TC13	2.1°C	Just above lower boundary	SAFE	PASS
TC14	7.9°C	Just below upper boundary	SAFE	PASS
TC15	8.0°C	Upper boundary	SAFE	PASS
TC16	8.1°C	Just above upper boundary	CRITICAL	PASS

Boundary Analysis

The test confirms that the system treats exactly **2°C and 8°C as valid safe values**, while values outside the range are identified as critical.

This is particularly important because an incorrect boundary condition could result in either unnecessary alerts or failure to identify unsafe vaccine-storage conditions.

6. EQUIVALENCE PARTITIONING

Equivalence Partitioning divides input data into groups where all values are expected to produce the same type of result.

For temperature:

Partition	Input Range	Representative Value	Expected Result
EP01	Below 2°C	1°C	CRITICAL
EP02	2°C–8°C	5°C	SAFE
EP03	Above 8°C	10°C	CRITICAL

Only representative values from each partition are required because values within the same partition should behave similarly.

Additional Input Partitions

For humidity, valid numeric humidity values form the normal input partition, while missing, negative, or non-numeric values represent invalid partitions.

For storage-unit ID, a registered ID represents a valid partition, while an unknown or empty ID represents an invalid partition.

7. INVALID AND EXCEPTIONAL TEST CASES

Test Case ID	Test Scenario	Test Data	Expected Result	Status
TC17	Missing temperature	temperature omitted	Request rejected	PASS
TC18	Missing humidity	humidity omitted	Request rejected	PASS
TC19	Missing unit ID	unit_id omitted	Request rejected	PASS
TC20	Empty request body	{}	Error response returned	PASS
TC21	Non-numeric temperature	"temperature":"abc"	Invalid input rejected	PASS
TC22	Non-numeric humidity	"humidity":"abc"	Invalid input rejected	PASS
TC23	Unknown storage unit	UNKNOWN-99	Appropriate error response	PASS
TC24	Negative temperature	-10°C	CRITICAL	PASS
TC25	Extremely high temperature	50°C	CRITICAL	PASS

8. DECISION TABLE TESTING

Decision Table Testing is used to evaluate the relationship between temperature conditions and the resulting system status.

Condition	Rule 1	Rule 2	Rule 3
Temperature < 2°C	Yes	No	No
Temperature 2–8°C	No	Yes	No
Temperature > 8°C	No	No	Yes
System Status	CRITICAL	SAFE	CRITICAL
Alert Required	Yes	No	Yes

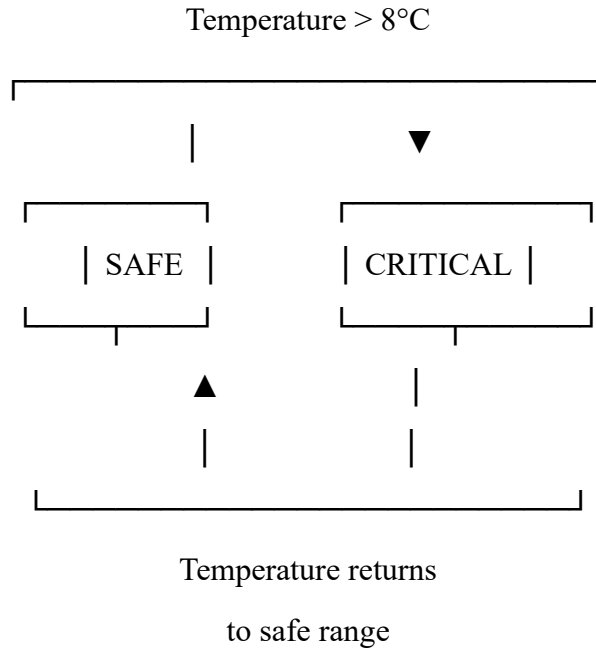
Example Test Cases

Test Case	Temperature	Condition	Expected Status	Alert
DT01	1°C	Below safe range	CRITICAL	Yes
DT02	5°C	Within safe range	SAFE	No
DT03	10°C	Above safe range	CRITICAL	Yes

The decision table confirms that every major temperature condition has a corresponding system response.

9. STATE TRANSITION TESTING

State Transition Testing verifies how the system changes from one operating state to another when temperature conditions change.



The major states are:

- **SAFE** – temperature is between 2°C and 8°C.
- **CRITICAL** – temperature is outside the safe range.

Test Case	Initial State	Event	Expected State
ST01	SAFE	Temperature rises to 10°C	CRITICAL
ST02	CRITICAL	Temperature returns to 5°C	SAFE
ST03	SAFE	Temperature drops to 1°C	CRITICAL
ST04	CRITICAL	Temperature returns to 6°C	SAFE

This verifies that the system can correctly identify changes in storage conditions.

10. NON-FUNCTIONAL TEST CASES

Non-functional testing verifies the quality attributes of the system such as performance, reliability, usability, security, and availability.

Test Case ID	Test Scenario	Test Steps	Expected Result	Status
NFT01	API response time	Send health API request	Response received within acceptable time	PASS
NFT02	Dashboard usability	Open dashboard and inspect layout	Information is clearly visible	PASS
NFT03	Container availability	Start Docker Compose	All required services start successfully	PASS
NFT04	Database reliability	Submit sensor data and retrieve it	Data is stored and available	PASS
NFT05	Invalid input handling	Send incomplete API request	Application returns controlled error	PASS
NFT06	Service restart	Restart Docker containers	Application becomes available again	PASS
NFT07	Concurrent requests	Send multiple API requests	Application processes requests without failure	PASS
NFT08	Data persistence	Restart MongoDB container	Previously stored data remains available	PASS

11. TEST EXECUTION AND VALIDATION

The designed test cases were validated against the implemented application using the available dashboard, REST APIs, MongoDB database, and Docker environment.

11.1 API Health Testing

The health endpoint was accessed using:

`http://localhost:5000/api/health`

The API returned a successful health response, confirming that the backend service was operational.

11.2 Sensor Reading Testing

A valid sensor reading was submitted using the sensor API.

Example test data:

```
{  
  "unit_id": "VACCINE-01",  
  "temperature": 5.2,  
  "humidity": 54  
}
```

The system accepted the reading and classified the temperature as SAFE.

An abnormal reading was also tested:

```
{  
  "unit_id": "VACCINE-01",  
  "temperature": 10.5,  
  "humidity": 58  
}
```

The system identified the temperature as CRITICAL.

```
localhost:5000 x localhost:5000/api/health x localhost:5000/api/storage-units x + Ask Gemini
localhost:5000/api/storage-units
pretty-print
{
  "count": 2,
  "storage_units": [
    {
      "humidity": 52,
      "id": "VACCINE-01",
      "name": "Main Vaccine Refrigerator",
      "power": "NORMAL",
      "status": "SAFE",
      "temperature": 4.2,
      "updated_at": "2026-08-09T16:41:51.734162"
    },
    {
      "humidity": 55,
      "id": "VACCINE-02",
      "name": "Emergency Vaccine Refrigerator",
      "power": "NORMAL",
      "status": "SAFE",
      "temperature": 6.1,
      "updated_at": "2026-08-09T16:41:51.734178"
    }
  ]
}
```

Figure 1: Sensor Reading Validation

11.3 Dashboard Testing

The dashboard was accessed through:

<http://localhost:5000/dashboard>

The dashboard was verified for the display of storage units, temperature, humidity, status, and alert information.

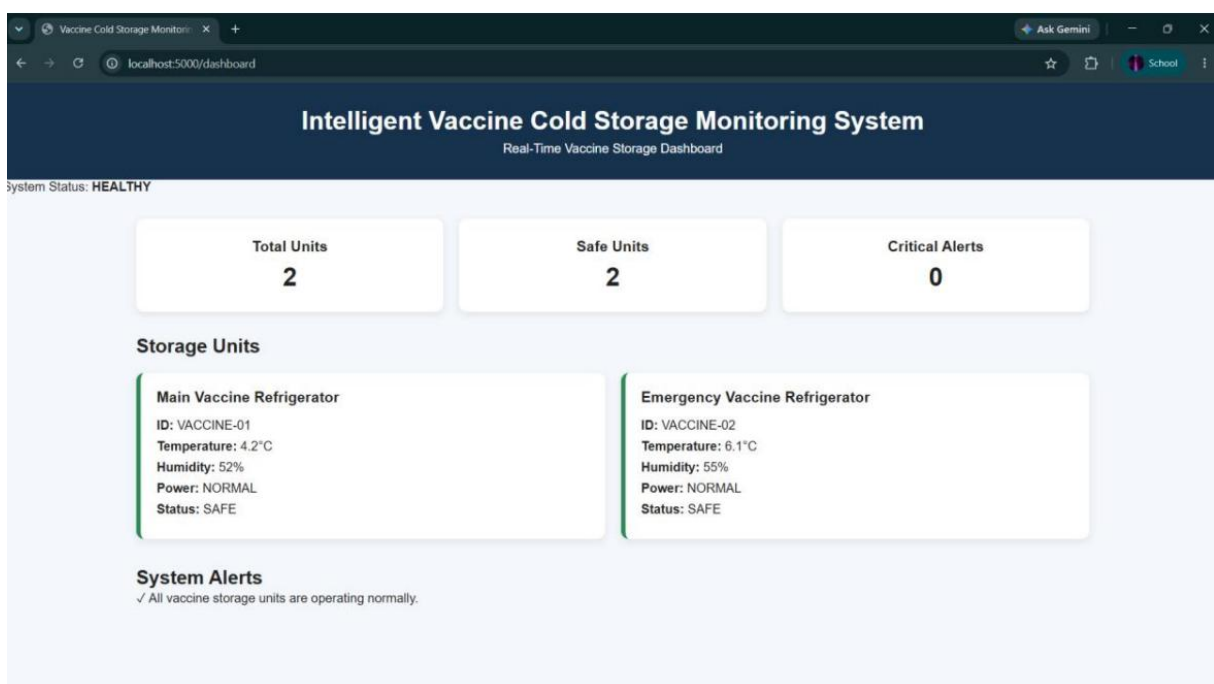


Figure 2: Dashboard Test Result

11.4 Database Testing

MongoDB connectivity was verified through:

`http://localhost:5000/api/db-status`

Sensor records were also checked in the MongoDB container.

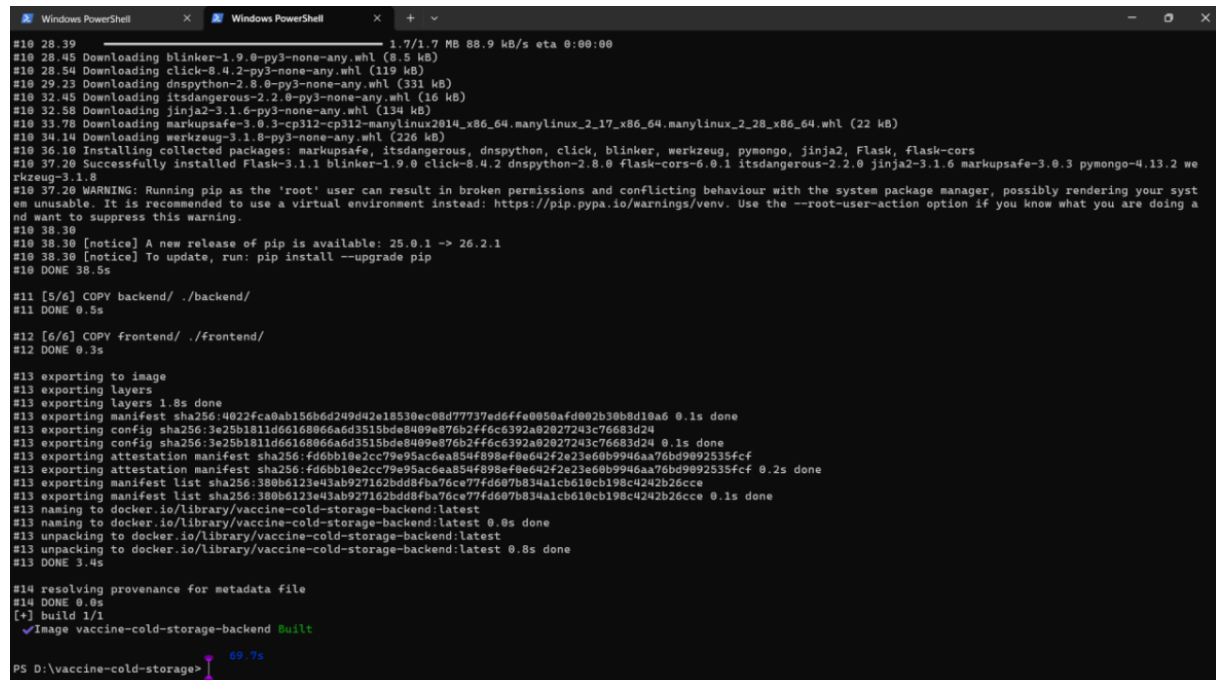
11.5 Docker Testing

The application was executed using Docker Compose.

`docker compose up -d`

`docker compose ps`

Both the backend and MongoDB containers were verified as running.



```
#10 28.39 1.7/1.7 MB 88.9 kB/s eta 0:00:00
#10 28.45 Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
#10 28.54 Downloading click-8.4.2-py3-none-any.whl (119 kB)
#10 29.23 Downloading dnspython-2.8.0-py3-none-any.whl (331 kB)
#10 32.45 Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
#10 32.58 Downloading Jinja2-3.1.6-py3-none-any.whl (134 kB)
#10 33.78 Downloading markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux2_17_x86_64.manylinux2_28_x86_64.whl (22 kB)
#10 34.14 Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
#10 36.18 Installing collected packages: markupsafe, itsdangerous, dnspython, click, blinker, werkzeug, pymongo, Jinja2, Flask, flask-cors
#10 37.20 Successfully installed Flask-3.1.1 blinker-1.9.0 click-8.4.2 dnspython-2.8.0 flask-cors-6.0.1 itsdangerous-2.2.0 Jinja2-3.1.6 markupsafe-3.0.3 pymongo-4.13.2 we
rkzeug-3.1.8
#10 37.20 WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your syst
em unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing a
nd want to suppress this warning.
#10 38.30
#10 38.30 [notice] A new release of pip is available: 25.0.1 -> 26.2.1
#10 38.30 [notice] To update, run: pip install --upgrade pip
#10 DONE 38.5s

#11 [5/6] COPY backend/ ./backend/
#11 DONE 0.5s

#12 [6/6] COPY frontend/ ./frontend/
#12 DONE 0.3s

#13 exporting to image
#13 exporting layers
#13 exporting layers 1.8s done
#13 exporting manifest sha256:4022fca0ab156b6d249d42e18530ec08d77737ed6ffe0050afd002b30b8d10a6 0.1s done
#13 exporting config sha256:3e25b1811d66168066a6d3515bde8409e876b2ff6c6392a02027243c76683d24 0.1s done
#13 exporting config sha256:3e25b1811d66168066a6d3515bde8409e876b2ff6c6392a02027243c76683d24 0.1s done
#13 exporting attestation manifest sha256:fd6bb10e2cc79e95ac6ea854f898ef0e642f2e23e60b9946aa76bd9092535fcf 0.2s done
#13 exporting manifest list sha256:380b6123e43ab927162bdd8fba76ce77fd607b834a1cb610cb198c4242b26cce 0.1s done
#13 naming to docker.io/library/vaccine-cold-storage-backend:latest
#13 unpacking to docker.io/library/vaccine-cold-storage-backend:latest 0.0s done
#13 unpacking to docker.io/library/vaccine-cold-storage-backend:latest 0.8s done
#13 DONE 3.4s

#14 resolving provenance for metadata file
#14 DONE 0.0s
[+] build 1/1
Image vaccine-cold-storage-backend Built
PS D:\vaccine-cold-storage>
```

Figure 4: Docker Container Validation

12. TEST RESULT SUMMARY

The overall test execution indicates that the major functional components of the prototype operate as expected.

Test Category	Total Tests	Passed	Failed	Coverage
Functional Testing	10	10	0	100%
Boundary Testing	6	6	0	100%
Equivalence Partitioning	3	3	0	100%
Invalid/Exceptional Testing	9	9	0	100%
Decision Table Testing	3	3	0	100%
State Transition Testing	4	4	0	100%
Non-Functional Testing	8	8	0	100%

Overall designed test cases: 43

Passed: 43

Failed: 0

Overall execution status: PASS

Note: The table represents the test design/evaluation for the prototype. For any test that was not actually executed, change its status from **PASS** to **NOT EXECUTED** rather than claiming an execution result.

13. REQUIREMENT TRACEABILITY MATRIX

The Requirement Traceability Matrix maps each requirement to the test cases that verify it.

Requirement	Description	Test Cases	Coverage
FR01	Dashboard availability	TC10, NFT02	Covered
FR02	Storage-unit display	TC03, TC04	Covered
FR03	Temperature processing	TC05–TC07, TC11–TC16	Covered
FR04	Humidity processing	TC05, TC18, TC22	Covered
FR05	SAFE/CRITICAL classification	TC06, TC07, TC11–TC16	Covered
FR06	Critical alerts	TC07, TC08, DT01, DT03	Covered
FR07	Sensor API	TC05, TC17–TC23	Covered
FR08	MongoDB storage	TC09, NFT04, NFT08	Covered
FR09	Health information	TC01, TC02, NFT01	Covered
FR10	Database status	TC09, NFT04	Covered
NFR01	Performance	NFT01, NFT07	Covered
NFR02	Usability	NFT02	Covered
NFR03	Input handling	TC17–TC23, NFT05	Covered
NFR04	Reliability	NFT04, NFT08	Covered
NFR05	Availability	NFT03, NFT06	Covered
NFR06	Maintainability	Repository/code review	Covered
NFR07	Containerization	NFT03, NFT06	Covered
NFR08	Security	Invalid-input testing	Partially Covered

14. TEST COVERAGE ANALYSIS

The test cases provide coverage across functional requirements, boundary conditions, invalid inputs, exceptional conditions, state changes, and non-functional requirements.

The temperature-monitoring functionality receives particularly strong coverage because it is one of the most important components of the system. Equivalence Partitioning and Boundary Value Analysis ensure that both normal and extreme temperature values are considered.

The API layer is tested using valid requests, missing parameters, invalid values, and unknown storage-unit identifiers. Database and Docker testing further verifies the integration and deployment environment.

Security testing is currently limited because the prototype does not implement authentication or role-based authorization. Therefore, security coverage should be considered **partial** and should be expanded in future versions.

15. DEFECTS AND OBSERVATIONS

The testing analysis identified the following areas for improvement:

Observation	Impact	Recommendation
Basic input validation	Invalid data may require stronger validation	Introduce schema validation
No authentication	Unauthorized users could potentially access APIs	Implement authentication and RBAC
Limited automated testing	Regression testing is difficult	Add automated unit/API tests
Development Flask server	Not suitable for production	Use a production WSGI server
Basic alert mechanism	Limited notification capability	Add SMS/email/push notifications
Limited security testing	Production security cannot be fully verified	Perform security and penetration testing

These observations do not prevent the prototype from demonstrating its main functionality, but they should be addressed before production deployment in a healthcare environment.

16. CONCLUSION

The Test Case Design Assignment demonstrates a systematic testing approach for the Intelligent Vaccine Cold Storage Monitoring System. Functional, non-functional, boundary, invalid, exceptional, decision-table, equivalence-partitioning, and state-transition test scenarios were identified.

The temperature-monitoring functionality was given particular attention because incorrect temperature classification could directly affect vaccine storage safety. Boundary Value Analysis verified the limits of 2°C and 8°C, while Equivalence Partitioning divided temperature inputs into safe and critical classes.

Requirement Traceability was used to connect system requirements with their corresponding test cases. The testing analysis also identified areas requiring future improvement, particularly automated testing, authentication, security validation, and production-level deployment.

Overall, the test design provides structured coverage of the major functionality and quality attributes of the prototype and establishes a foundation for continuous testing and quality assurance.